ITERAZIONE 3

2.1 INTRODUZIONE

Terminata la seconda iterazione, si procede con la terza ed ultima iterazione. L'obiettivo di iterazione sarà quello di affinare la Visione, sviluppare progressivamente il nucleo dell'architettura del software, affrontare i principali rischi identificati, definire la maggior parte dei requisiti e dell'ambito del progetto, oltre a fornire stime più accurate sul piano di lavoro e sulle risorse complessive necessarie.

2.2 CASI D'USO UC7, UC8, UC9, UC10

I casi d'uso costituiscono uno strumento essenziale per descrivere le interazioni tra gli attori e il sistema, mettendo in evidenza i flussi operativi necessari per raggiungere specifici obiettivi funzionali. Mediante una descrizione organizzata e approfondita, ogni caso d'uso delinea il contesto, i requisiti funzionali e i comportamenti attesi del sistema in relazione alle azioni eseguite dagli attori coinvolti.

L'approccio adottato si focalizza sull'identificazione dei processi chiave legati alla gestione di un B&B, assicurando una rappresentazione chiara e completa delle funzionalità disponibili. Ogni caso d'uso prende in considerazione sia gli scenari principali che le eventuali situazioni eccezionali, garantendo che il sistema sia in grado di gestire efficacemente anche contesti non ordinari. I casi d'uso affrontati nel dettaglio, sono già sono stati descritti nel pdf Modello dei casi d'uso nella stessa cartella.

2.3 ANALISI ORIENTATA AGLI OGGETTI

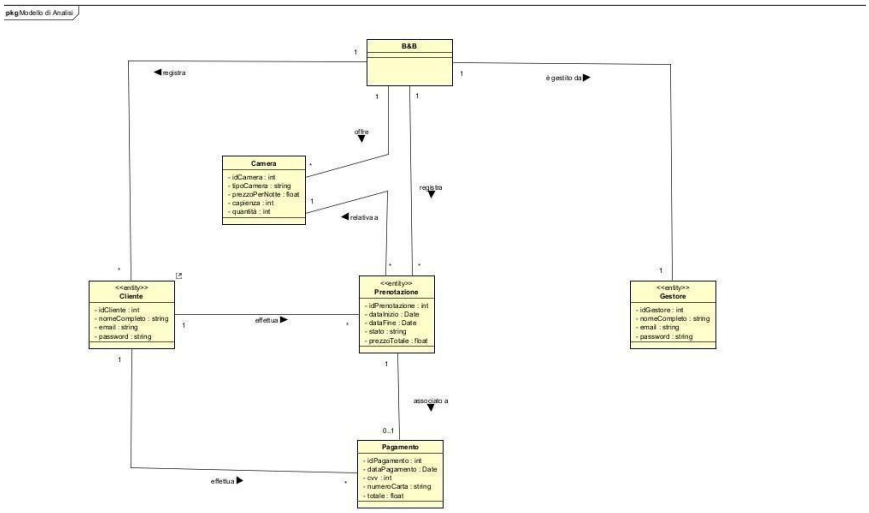
L'analisi orientata agli oggetti si basa sulla creazione di una descrizione del dominio da un punto di vista ad oggetti. Vengono utilizzati diversi strumenti per fornire tale descrizione: Modello di Dominio, SSD (Sequence System Diagram) e Contratti delle operazioni.

2.3.1 MODELLO DI DOMINIO

È stato possibile identificare le seguenti classi concettuali:

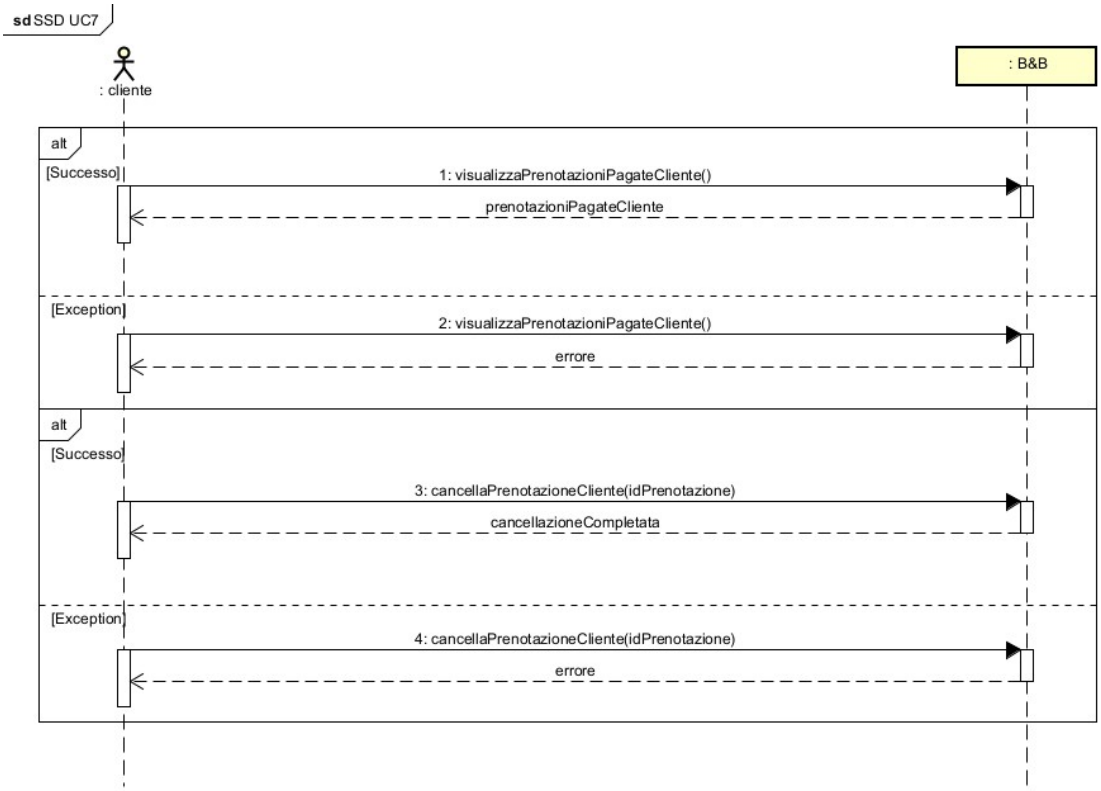
- B&B: che rappresenta il sistema B&B;
- Gestore: attore primario, che interagisce direttamente con il sistema;
- Prenotazione: contiene tutti i dettagli della prenotazione effettuata dal cliente;
- Cliente: cliente del B&B che effettua prenotazioni di camere;
- Camera: contiene le informazioni delle camere del B&B;
- Pagamento: contiene tutti i dettagli del pagamento.

Da cui, tenendo conto di associazioni e attributi, è stato ricavato il seguente Modello di Dominio:

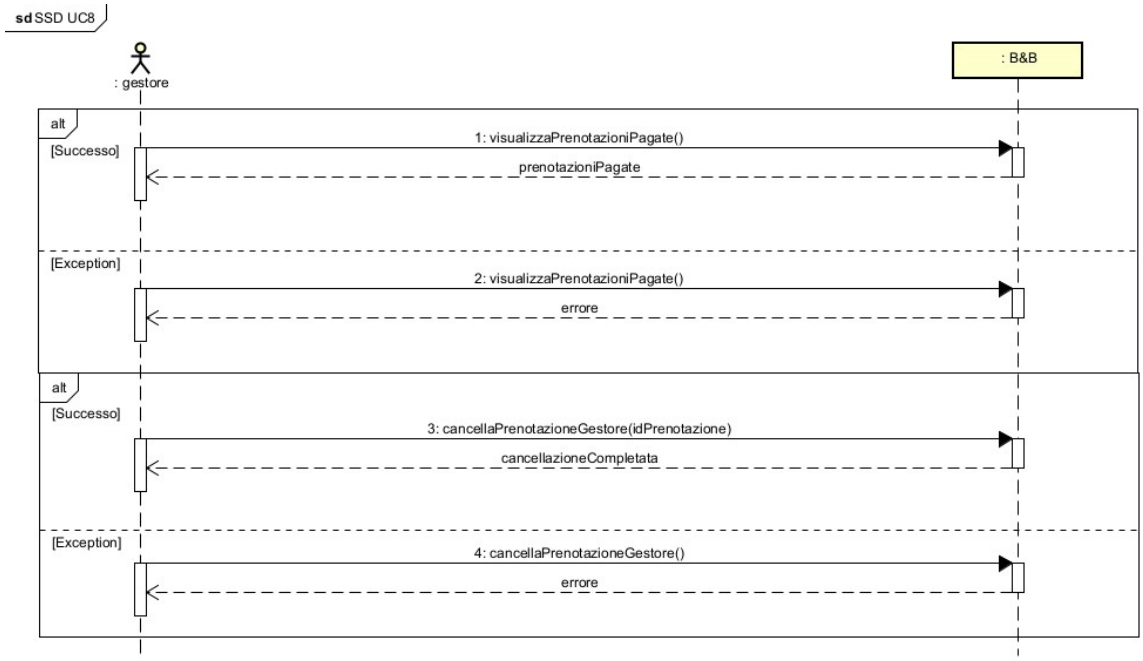


2.3.2 DIAGRAMMI DI SEQUENZA DI SISTEMA

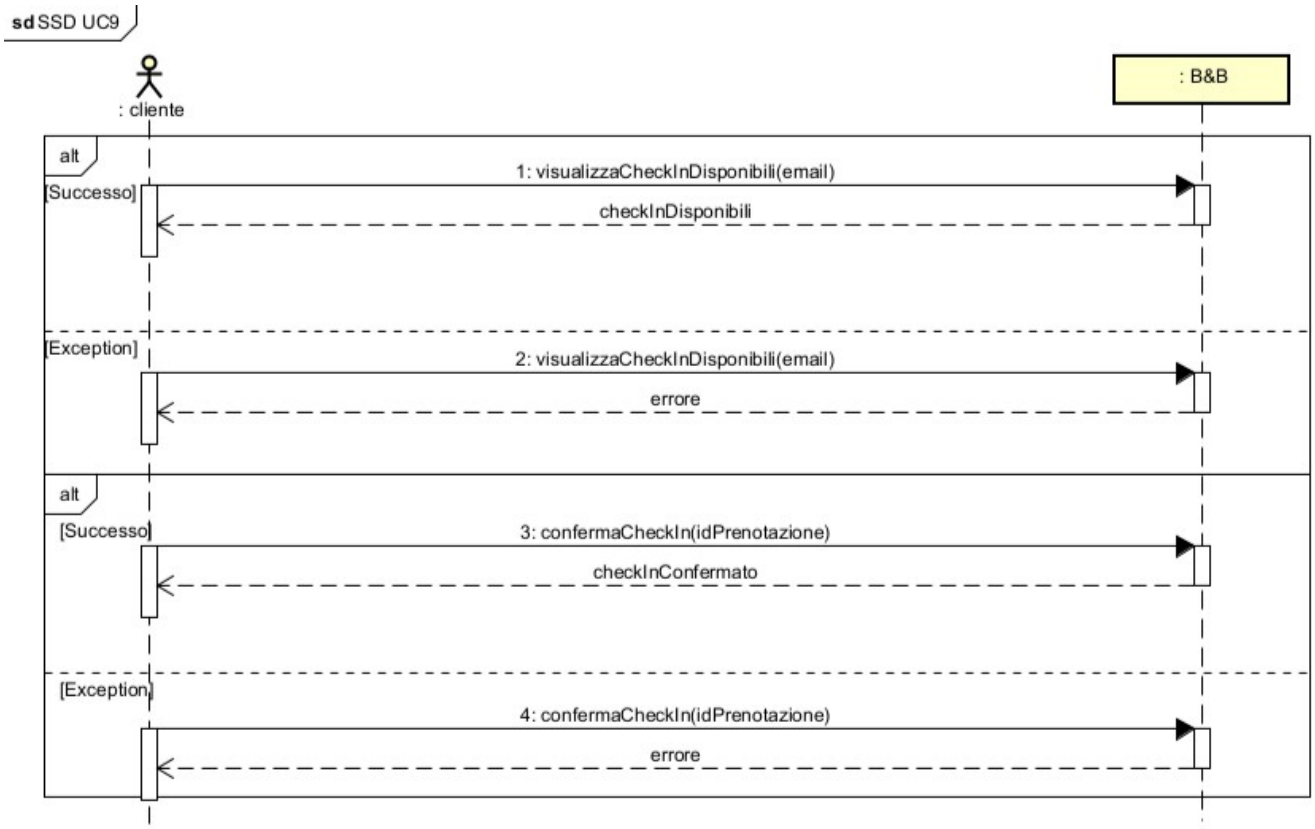
UC7: Annullamento autonomo della prenotazione



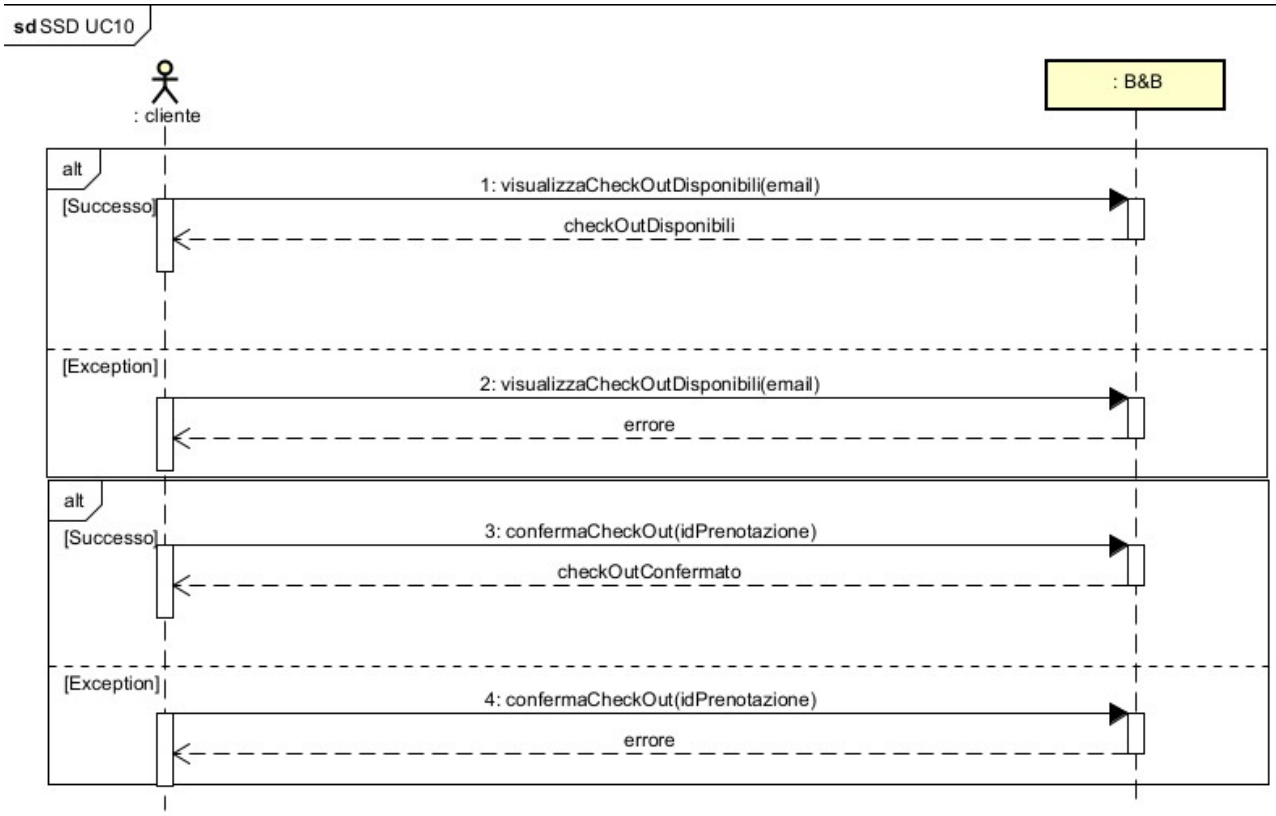
UC8: Annullamento della prenotazione da parte del gestore



UC9: Check-in



UC10: Check-out



2.3.3 Contratti delle operazioni

Vengono ora descritte attraverso i Contratti le principali operazioni di sistema che si occupano di gestire gli eventi di sistema individuati negli SSD.

Operazione: visualizzaPrenotazioniPagateCliente()

Riferimenti: Caso d'uso UC4

Pre-Condizioni:

-Il cliente c ha effettuato l'accesso al sistema

-Il cliente c ha effettuato e pagato almeno una prenotazione

Post-Condizioni: Vengono mostrate tutte le prenotazioni con stato= "PAGATA" associate al cliente c

Operazione:cancellaPrenotazioneCliente(idPrenotazione)

Riferimenti: Caso d'uso UC7

Pre-Condizioni:

- Il cliente c ha effettuato l'accesso al sistema
- c sceglie una prenotazione associata ad esso da cancellare

Post-Condizioni:

- la PRENOTAZIONE p associata al cliente c aggiorna lo stato in "CANCELLATA"

Operazione: visualizzaPrenotazioniPagate()

Riferimenti: Caso d'uso UC8

Pre-Condizioni:

-Il Gestore g ha effettuato l'accesso al sistema Post-Condizioni:

-Vengono mostrate tutte le prenotazioni con stato "PAGATA"

Operazione:cancellaPrenotazioneGestore(idPrenotazione)

Riferimenti: Caso d'uso UC8

Pre-Condizioni:

- cancellazione della prenotazione p da parte del gestore g

Post-Condizioni:

- la PRENOTAZIONE p aggiorna lo stato in "ELIMINATA"

Operazione: visualizzaCheckInDisponibili(email)

Riferimenti: Caso d'uso UC9

Pre-Condizioni:

-Il cliente c ha effettuato l'accesso al sistema

-Esiste almeno una prenotazione associata al cliente c con stato "PAGATA"

Post-Condizioni: vengono mostrate tutte le PRENOTAZIONI con stato "PAGATA" associate al cliente c

Operazione: confermaCheckIn(idPrenotazione)

Riferimenti: Caso d'uso UC9

Pre-Condizioni: il cliente ha effettuato l'accesso al sistema ed ha selezionato una prenotazione p associata a lui con stato "PAGATA" per effettuare il check-in Post-Condizioni:

- La prenotazione p aggiorna lo stato in "CHECK-IN"

Operazione: visualizzaCheckOutDisponibili(email)

Riferimenti: Caso d'uso UC10

Pre-Condizioni:

-Il cliente c ha effettuato l'accesso al sistema

-Esiste almeno una prenotazione associata al cliente c con stato "CHECK-IN"

Post-Condizioni: vengono mostrate tutte le PRENOTAZIONI con stato "CHECK-IN" associate al cliente c con dataInizio.before(LocalDate)=true

Operazione: confermaCheckOut(idPrenotazione)

Riferimenti: Caso d'uso UC10

Pre-Condizioni: il cliente ha effettuato l'accesso al sistema ed ha selezionato una prenotazione p associata a lui con stato "CHECK-IN" per effettuare il check-out Post-Condizioni:

- La prenotazione p aggiorna lo stato in "CHECK-OUT"

2.4 Progettazione

La progettazione orientata agli oggetti è la disciplina di UP interessata alla definizione degli oggetti software, delle loro responsabilità e a come questi collaborano per soddisfare i requisiti individuati nei passi precedenti. L'elaborato principale di questa fase che è stato preso in considerazione è il Modello di Progetto, ovvero l'insieme dei diagrammi che descrivono la progettazione logica sia da un punto di vista dinamico (Diagrammi di Interazione) che da un punto di vista statico (Diagramma delle Classi). Seguono dunque i diagrammi di Interazione più significativi e il diagramma delle Classi relativi ai casi d'uso determinati a seguito di un attento studio degli elaborati scritti in precedenza.

2.4.1 Refactoring

Nel corso della terza iterazione, il focus del team di sviluppo si è concentrato sul **refactoring** dell'architettura del sistema. All'aumentare delle funzionalità e della complessità, si è resa necessaria una riorganizzazione strutturale per evitare l'insorgere di *code smell* (come classi troppo grandi o logica condizionale eccessiva) e per garantire una maggiore flessibilità futura.

A tal fine, sono stati introdotti e implementati alcuni dei principali **Design Pattern GoF (Gang of Four)** e pattern **GRASP**. Questa evoluzione architetturale ha richiesto un fisiologico adattamento dei diagrammi prodotti nelle iterazioni precedenti (Design Class Diagram e Sequence Diagram), al fine di rispecchiare in modo accurato la nuova distribuzione delle responsabilità.

Di seguito si descrivono i pattern utilizzati, il loro significato teorico e lo specifico utilizzo all'interno del nostro dominio applicativo:

Singleton Pattern (Pattern Creazionale)

- **Significato:** Il Singleton garantisce che di una determinata classe venga creata una e una sola istanza all'interno dell'intero sistema, fornendo un punto di accesso globale ad essa.
- **Utilizzo nel sistema:** È stato applicato alla classe principale BnB. Poiché il sistema deve gestire la persistenza in memoria (tramite mappe) e lo smistamento delle richieste per un'unica struttura ricettiva, permettere l'istanziazione di più oggetti BnB avrebbe portato a un'inconsistenza fatale dei dati. Attraverso un costruttore privato e il metodo statico getInstance(), abbiamo centralizzato in modo sicuro l'accesso al controller principale.

Facade Pattern (Pattern Strutturale)

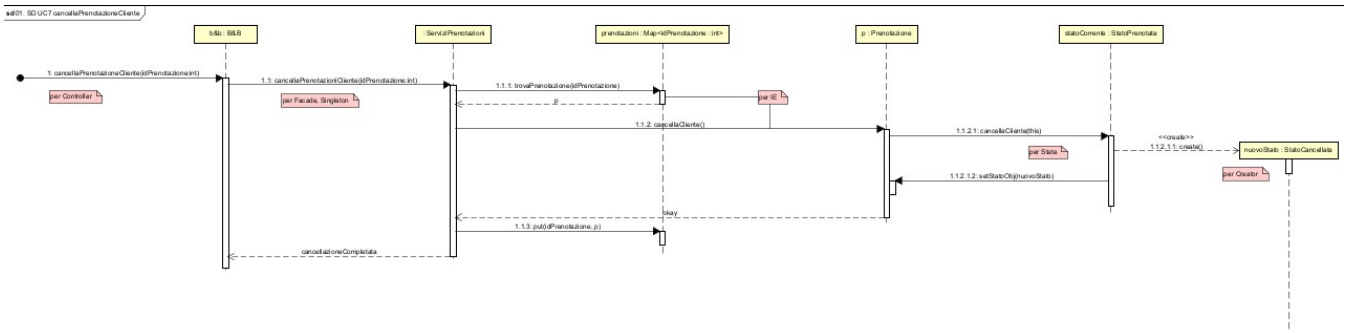
- **Significato:** Il Facade fornisce un'interfaccia unificata e di alto livello per interfacciarsi con un insieme di interfacce complesse all'interno di un sottosistema, nascondendone i dettagli implementativi e riducendo l'accoppiamento.
- **Utilizzo nel sistema:** La classe BnB assume il ruolo di Facade. Il livello di presentazione (il Main / UI) non interagisce mai direttamente con le strutture dati (le HashMap) o con le logiche interne di gestione. Il Main invoca metodi semplificati esposti dal Facade (es. bnb.pagaRichiesta(...)), il quale si occupa internamente di orchestrare i vari sottosistemi (ServiziPrenotazioni, ServiziPagamenti, ServiziClienti). Questo garantisce il principio di *Information Hiding* e un disaccoppiamento totale tra l'interfaccia utente e la logica di business.

State Pattern (Pattern Comportamentale)

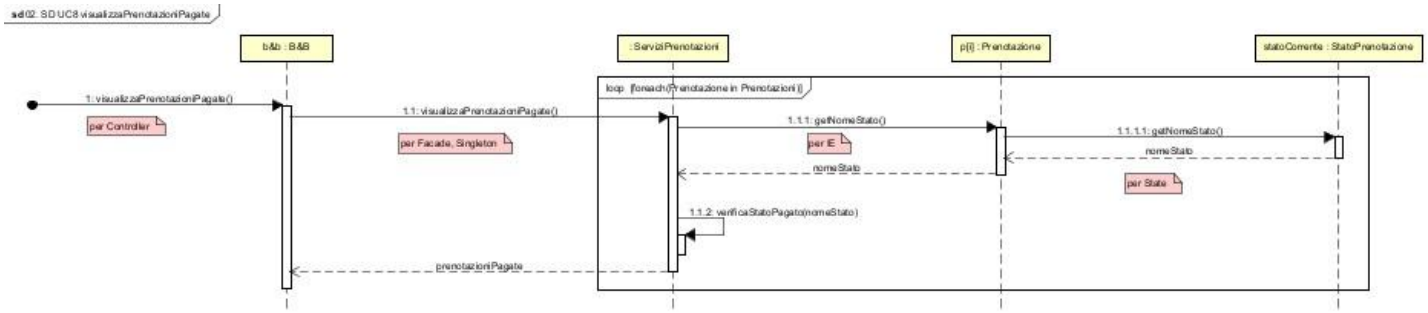
- **Significato:** Lo State Pattern permette a un oggetto (chiamato *Context*) di alterare il proprio comportamento quando il suo stato interno cambia. L'oggetto sembrerà cambiare classe dinamicamente, evitando l'uso di complessi e fragili costrutti condizionali (if/else o switch).
- **Utilizzo nel sistema:** È stato il refactoring più importante e ha riguardato la gestione del ciclo di vita della classe Prenotazione (il Context). Inizialmente, i controlli per verificare se una prenotazione potesse essere pagata, approvata o cancellata richiedevano controlli incrociati sulle stringhe di stato. Con il pattern State, abbiamo creato un'interfaccia StatoPrenotazione e classi concrete per ogni stato (es. StatoAttesa, StatoPrenotata, StatoCheckIn). La prenotazione ora *delega* l'esecuzione delle operazioni al suo stato corrente. Questo ha reso il codice pulito e robusto, permettendo l'aggiunta di futuri stati senza dover modificare il codice esistente.

2.4.2 Diagrammi di Sequenza

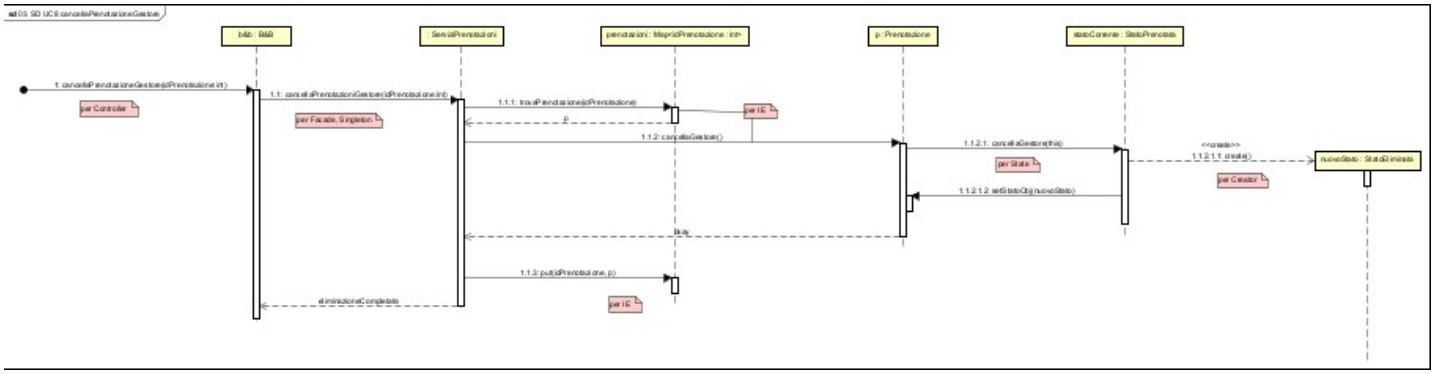
- Cancellazione della Prenotazione da parte del Cliente



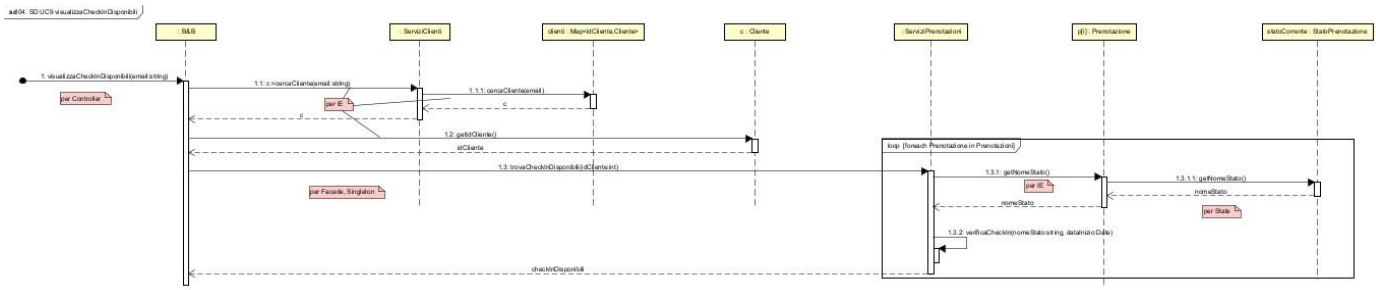
- Visualizzazione delle prenotazioni pagate



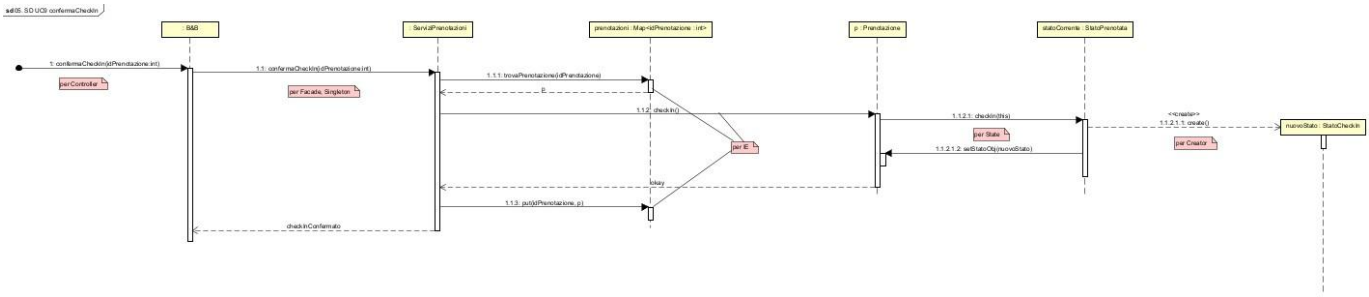
- Cancellazione della prenotazione da parte del gestore



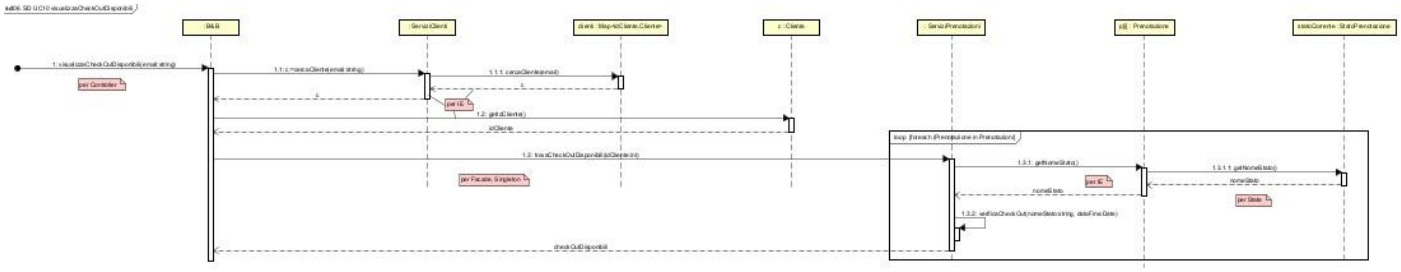
- Visualizzazione dei CheckIn disponibili



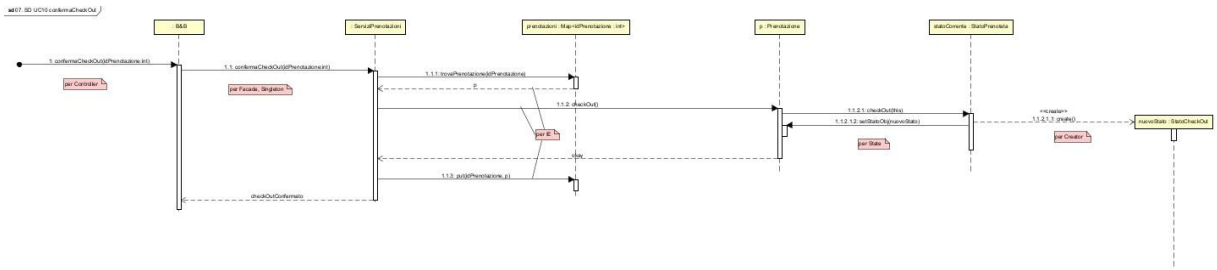
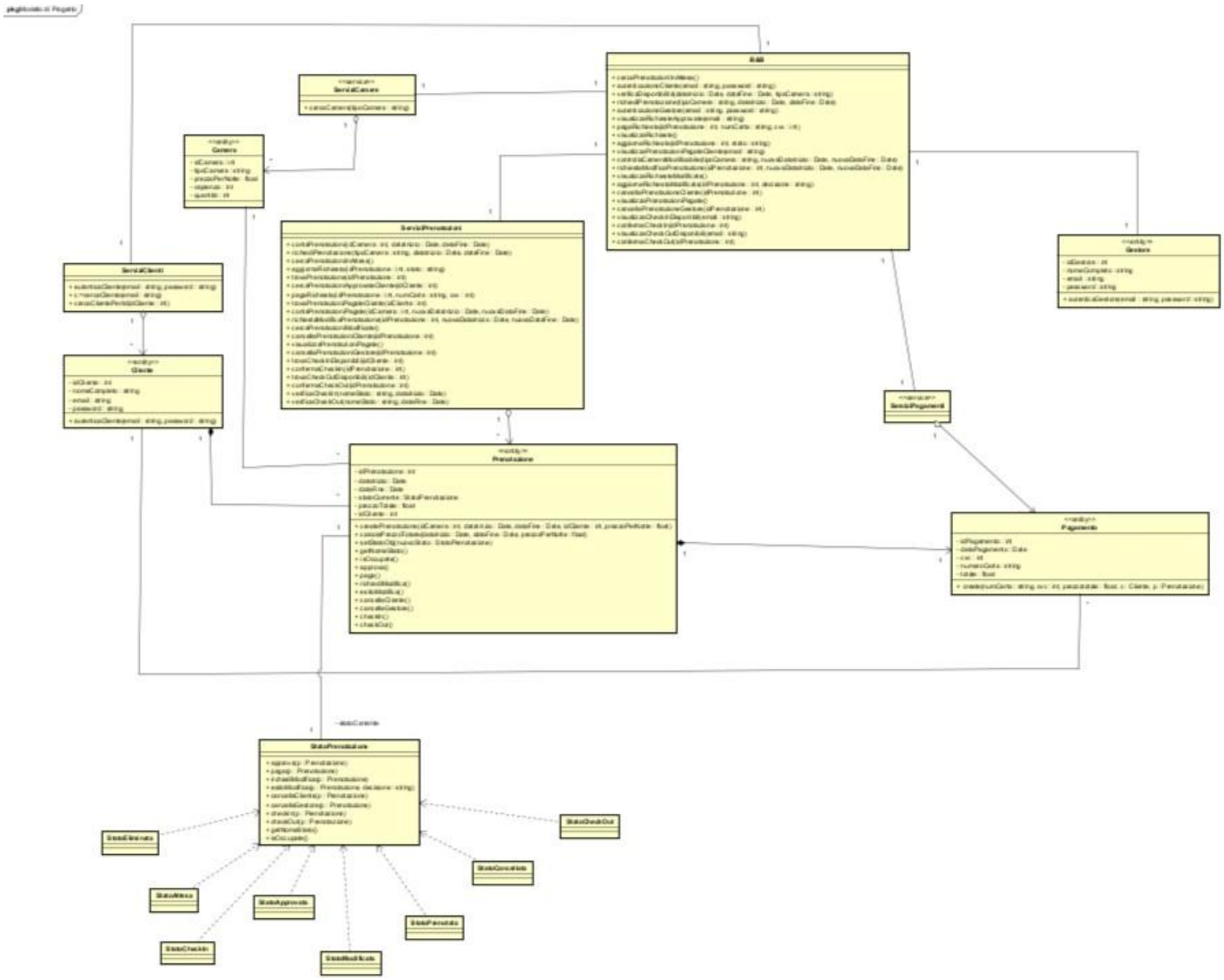
- Conferma di un CheckIn



- Visualizzazione dei CheckOut disponibili



- Conferma di un CheckOut



2.4.3 Diagramma delle classi